PROGRAMMING 742

RICHFIELD
richfield.ac.za

Name and Surname: Obakeng Mokgethwa Mokoena

Student ITS No: 402111288

Qualification: BSCIT **Year of Study:** 3 **Semester:** 2

Assignment due date: 23/10/2026 **Date submitted:** 13/10/20

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

- ☐ Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
- ☐ Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

- ☐ Table of Contents
- ☐ Introduction
- ☐ Main Body (with relevant subheadings)
- ☐ Conclusion
- ☐ References
- ☐ Submit the assignment in PDF format on Moodle.
- ☐ Use the specified cover page provided.
- ☐ Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: **Date:** 13/10/2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Table of Contents

INTRODUCTION 3
MAIN BODY 3
User Authentication and Role Logic 3
Admin Dashboard 8
Host Dashboard 10
Application Process 12
Booking And Check-In 15
Notifications and Messaging 19
Data and Architechture 21
REVIEWS 23
ADDITIONAL FEATURES 23
WISHLISTITEM 24
REFERENCE 26

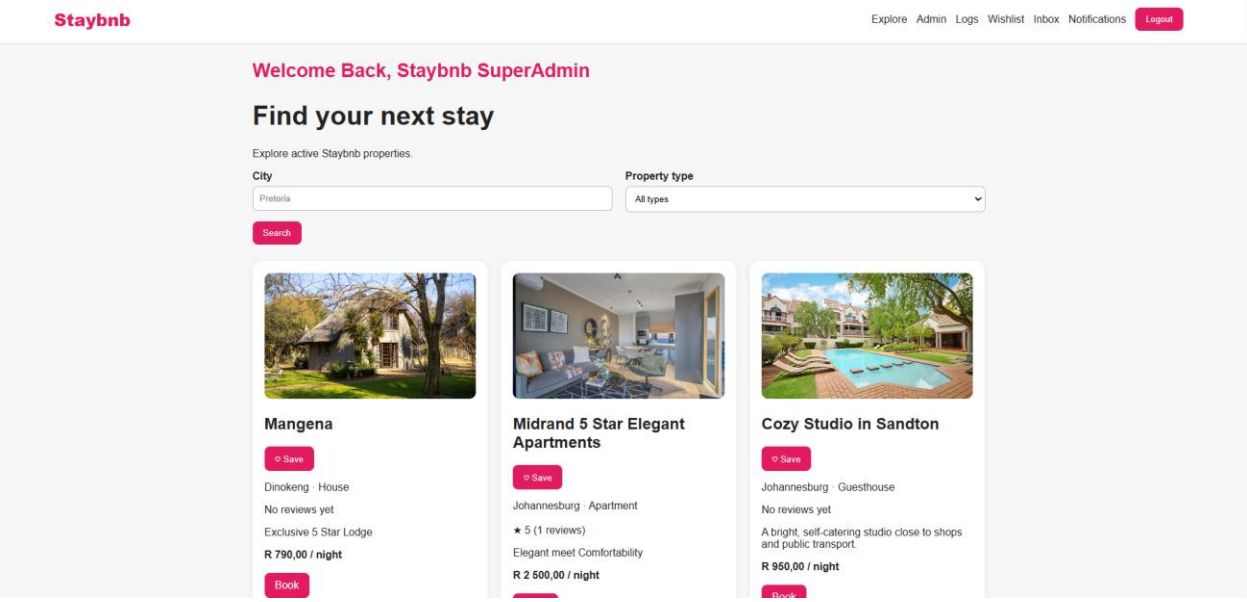
INTRODUCTION

In order to meet the criteria of this assignement, this report details the design, development, and testing of Staybnb, a managed property rental platform created with ASP.NET Core MVC, Entity Framework Core, and SQL Server. The system simulates a short-term rental marketplace's whole lifecycle: a visitor registers as a guest, submits property details to apply to become a host, and, if accepted, is given access to a host-specific dashboard for managing listings and reservations. Automatically seeded on the first run, administrators manage to platform by analyzing system activity, evaluating host apps, and elevating trusted users to admin.

MAIN BODY

User Authentication and Role Logic

A SuperAdmin account Is programmatically seeded upon application starting, and all new registrants are automatically assigned the Guest role upon sign-up. The system adds the Host role to the applicants's account and removes the Guest role when an Admin approves a pending Host application. This ensures that a user is never both a Guest and a Host at the same time. The [Authoize(Roles==")] element on controllers is used to impose role-basef access control throughout the application, preventing a Host from accessing Guest-onlt and vise versa.




```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Staybnb.Data;
using Staybnb.Models;
using Staybnb.Services;

namespace Staybnb.Controllers;
[Authorize(Roles="Admin,SuperAdmin")]
public class AdminController : Controller
{
    readonly ApplicationDbContext _db; readonly UserManager<ApplicationUser> _users;
readonly IActivityLogger _log;
    public AdminController(ApplicationDbContext db,UserManager<ApplicationUser>
users,IActivityLogger log){_db=db;_users=users;_log=log;}
    public async Task<IActionResult> Dashboard()
```

```
    {
        var apps = await _db.HostApplications.Include(x => x.Applicant).OrderByDescending(x
=> x.Id).ToListAsync();
        ViewBag.TotalUsers = await _users.Users.CountAsync();
        ViewBag.PendingHostApps = apps.Count(a => a.Status == ApplicationStatus.Pending);
        ViewBag.ActiveProperties = await _db.Properties.CountAsync(p => p.IsActive);
        ViewBag.PlatformRevenue = await _db.Bookings.Where(b => b.Status ==
BookingStatus.Approved || b.Status == BookingStatus.CheckedIn).SumAsync(b =>
b.TotalPrice);
        ViewBag.TotalBookings = await _db.Bookings.CountAsync();
        ViewBag.ApprovedHosts = (await _users.GetUsersInRoleAsync("Host")).Count;
        return View(apps);
    }
    [HttpPost,ValidateAntiForgeryToken] public async Task<IActionResult> Decide(int id,bool
approve)
    {
        var a=await _db.HostApplications.FirstOrDefaultAsync(x=>x.Id==id);if(a==null)return
NotFound();
        if(a.Status!=ApplicationStatus.Pending)return BadRequest();
        a.Status=approve?ApplicationStatus.Approved:ApplicationStatus.Rejected;
        if(approve){
            var user=await _users.FindByIdAsync(a.ApplicantId); if(user==null)return NotFound();
            await _users.AddToRoleAsync(user,"Host"); await
_users.RemoveFromRoleAsync(user,"Guest");
            var p=new
Property{HostId=user.Id,Title=a.Title,Description=a.Description,City=a.City,Address=a.Addres
s,PropertyType=a.PropertyType,PricePerNight=a.PricePerNight,CleaningFee=a.CleaningFee,Ser
viceFee=a.ServiceFee,IsActive=true};
            _db.Properties.Add(p); await _db.SaveChangesAsync();
            _db.PropertyImages.Add(new
PropertyImage{PropertyId=p.Id,ImagePath=a.ImagePath});
```

```
            _db.Notifications.Add(new Notification{UserId=user.Id,Title="Host Application
Approved",Message="Your property application has been approved. You are now a Host."});
        } else _db.Notifications.Add(new Notification{UserId=a.ApplicantId,Title="Host
Application Rejected",Message="Your host application was rejected."});
        await _db.SaveChangesAsync(); await _log.LogAsync(_users.GetUserId(User),"Host
application decision",$"Application {id}: {a.Status}");
        return RedirectToAction(nameof(Dashboard));
    }
    [Authorize(Roles="SuperAdmin")]
    public async Task<IActionResult> Users()=>View(await
_users.GetUsersInRoleAsync("Guest"));
    [Authorize(Roles="SuperAdmin"),HttpPost,ValidateAntiForgeryToken]
    public async Task<IActionResult> Promote(string id)
    {
        var user=await _users.FindByIdAsync(id);if(user==null)return NotFound();
        await _users.AddToRoleAsync(user,"Admin"); await
_users.RemoveFromRoleAsync(user,"Guest");
        await _log.LogAsync(_users.GetUserId(User),"Role change",$"User {id} promoted to
Admin");
        return RedirectToAction(nameof(Users));
    }
    public async Task<IActionResult> Logs()=>View(await
_db.ActivityLogs.OrderByDescending(x=>x.CreatedAt).Take(500).ToListAsync());
}
```

```
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.EntityFrameworkCore;
using Staybnb.Data;
using Staybnb.Models;

namespace Staybnb.Services;

public class StaybnbSeeder
{
   private readonly UserManager<ApplicationUser> _users;
   private readonly RoleManager<IdentityRole> _roles;
   private readonly ApplicationDbContext _db;

   public StaybnbSeeder(UserManager<ApplicationUser> users, RoleManager<IdentityRole>
roles, ApplicationDbContext db)
   { _users = users; _roles = roles; _db = db; }

   public async Task SeedAsync()
   {
      string[] roleNames = ["Guest", "Host", "Admin", "SuperAdmin"];
      foreach (var role in roleNames)
         if (!await _roles.RoleExistsAsync(role)) await _roles.CreateAsync(new
IdentityRole(role));

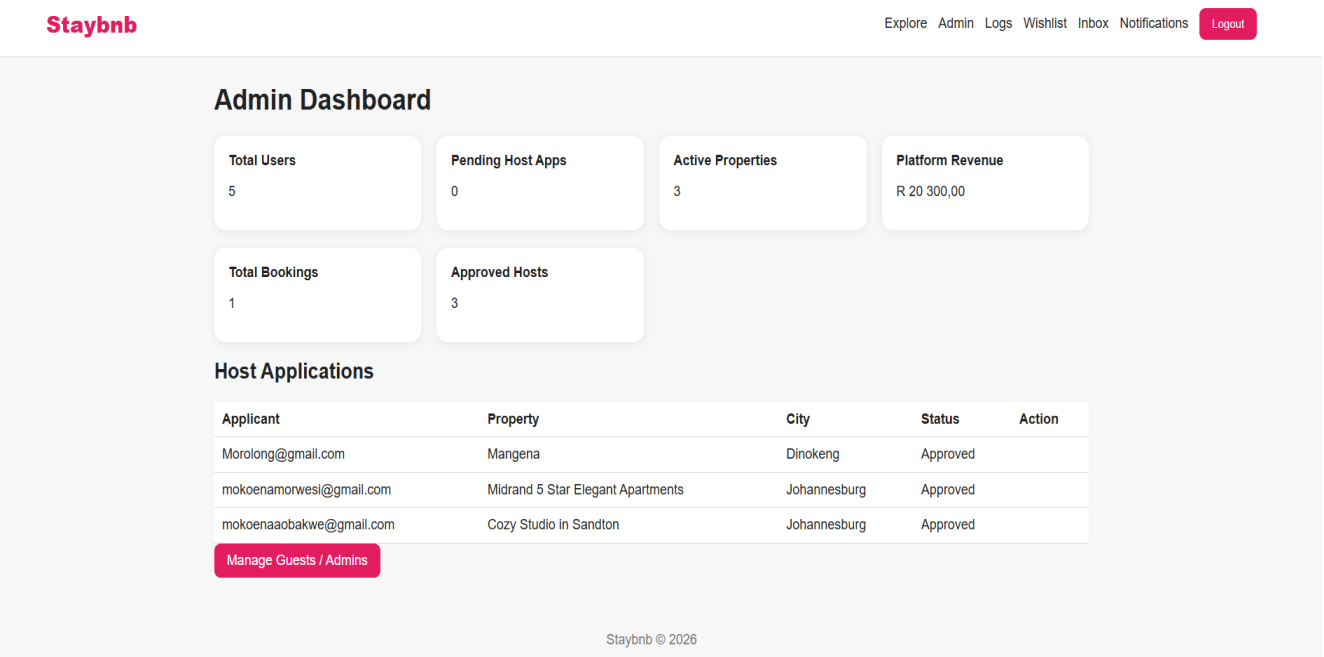
      const string email = "superadmin@staybnb.local";
      var admin = await _users.FindByEmailAsync(email);
      if (admin == null)
      {
         admin = new ApplicationUser { UserName = email, Email = email, EmailConfirmed =
true, FullName = "Staybnb SuperAdmin" };
         var result = await _users.CreateAsync(admin, "Admin123!");
         if (!result.Succeeded) throw new Exception(string.Join("; ", result.Errors.Select(e =>
```

```
e.Description)));
        }
        if (!await _users.IsInRoleAsync(admin, "SuperAdmin"))
            await _users.AddToRoleAsync(admin, "SuperAdmin");

        if (!await _db.Properties.AnyAsync())
        {
            _db.Properties.Add(new Property
            {
                Title = "Sample Staybnb Apartment", Description = "A sample property for testing.",
                City = "Pretoria", Address = "Pretoria, Gauteng", PropertyType = "Apartment",
                PricePerNight = 850, CleaningFee = 150, ServiceFee = 100, IsActive = true, HostId =
admin.Id
            });
            await _db.SaveChangesAsync();
        }
    }
}
```

Admin Dashboard

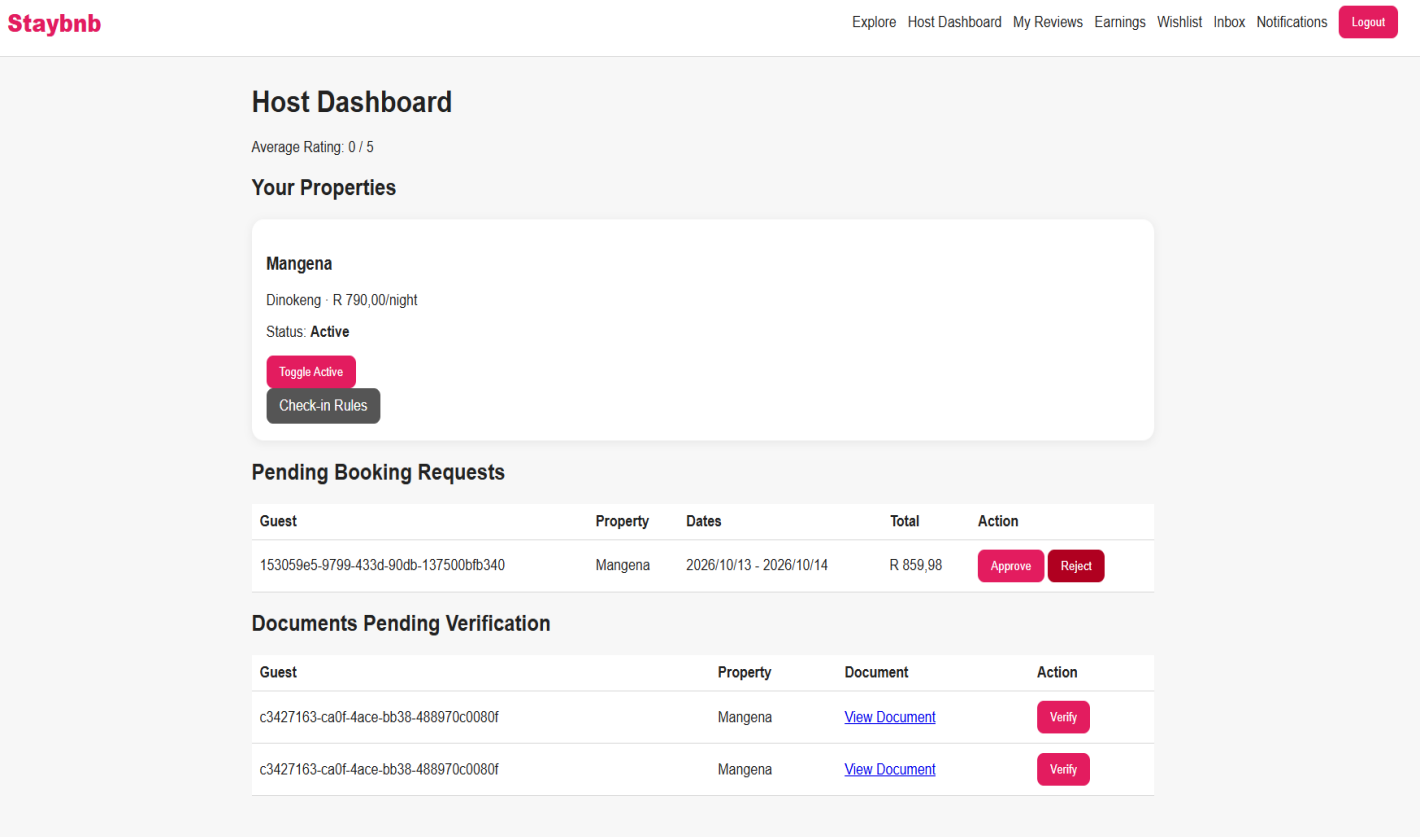
A comprehensive summary of platform activities, including total users, pending host applications, active properties, platform revenue, total bookings and the number of approved hosts, can be found on the Admin Dashboard. From here, a SuperAdmin can elevate any Guest to the Admin position and an Admin can accept or reject Host applications. Every logged system action is listed on the Activity Logs page, providing a complete audit trail of role changes, logins and booking status updates.




```
public async Task<IActionResult> Dashboard()
{
    var apps = await _db.HostApplications.Include(x => x.Applicant).OrderByDescending(x =>
x.Id).ToListAsync();
    ViewBag.TotalUsers = await _users.Users.CountAsync();
    ViewBag.PendingHostApps = apps.Count(a => a.Status == ApplicationStatus.Pending);
    ViewBag.ActiveProperties = await _db.Properties.CountAsync(p => p.IsActive);
    ViewBag.PlatformRevenue = await _db.Bookings.Where(b => b.Status ==
BookingStatus.Approved || b.Status == BookingStatus.CheckedIn).SumAsync(b =>
b.TotalPrice);
    ViewBag.TotalBookings = await _db.Bookings.CountAsync();
    ViewBag.ApprovedHosts = (await _users.GetUsersInRoleAsync("Host")).Count;
    return View(apps);
}
```

Host Dashboard

A host-specific dashboard is accessible to the user after a host application is accepted. From here, they can determine the necessary check-in guidelines and paperwork for each property, examine and accept or reject outstanding booking requests, and change a property's active status. A 'Documents Pending Verification' section with a direct Verify action was added to close a real gap between the backend logic and the user interface after it was found during testing that the document-verification feature was present in the controller but had no corresponding link anywhere in the Host Dashboard view.



```
public async Task<IActionResult> BookingDecision(int id, bool approve)
 {
    var uid=_users.GetUserId(User)!; var b=await
_db.Bookings.Include(x=>x.Property).FirstOrDefaultAsync(x=>x.Id==id &&
x.Property!.HostId==uid);
    if(b==null) return NotFound();
b.Status=approve?BookingStatus.Approved:BookingStatus.Rejected;
    _db.Notifications.Add(new Notification{UserId=b.GuestId,Title=$"Booking
{(approve?"Approved":"Rejected")}",Message=$"Your booking for {b.Property!.Title} has been
{(approve?"approved":"rejected")}."});
    await _db.SaveChangesAsync(); await _log.LogAsync(uid,"Booking status
update",$"Booking {id}: {b.Status}");
    return RedirectToAction(nameof(Dashboard));
 }
 [HttpGet] public async Task<IActionResult> CheckIn(int propertyId)

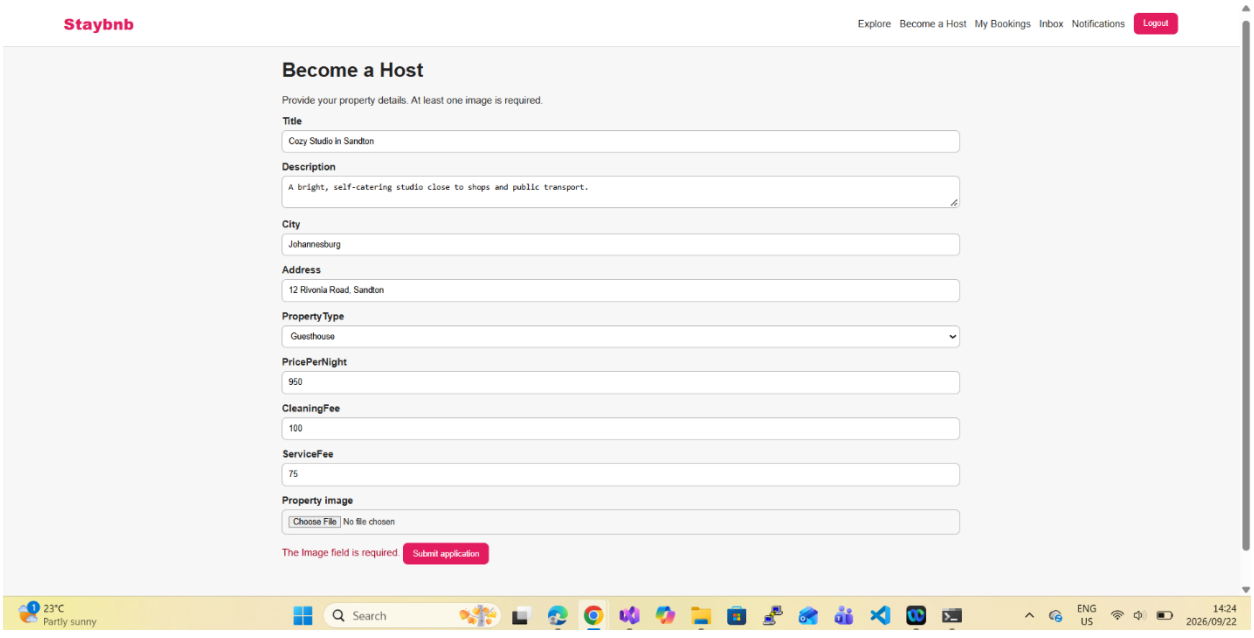


   [HttpGet] public async Task<IActionResult> VerifyDocument(int id)
   {
      var uid=_users.GetUserId(User)!; var d=await
_db.GuestDocuments.Include(x=>x.Booking).ThenInclude(b=>b!.Property).FirstOrDefaultAsyn
c(x=>x.Id==id&&x.Booking!.Property!.HostId==uid);
      if(d==null) return NotFound(); return View(d);
   }
   [HttpPost, ValidateAntiForgeryToken] public async Task<IActionResult>
VerifyDocument(int id,bool verified)
   {
      var uid=_users.GetUserId(User)!; var d=await
_db.GuestDocuments.Include(x=>x.Booking).ThenInclude(b=>b!.Property).FirstOrDefaultAsyn
```

```
c(x=>x.Id==id&&x.Booking!.Property!.HostId==uid);
        if(d==null) return NotFound(); d.Verified=verified; if(verified)
d.Booking!.Status=BookingStatus.CheckedIn;
        await _db.SaveChangesAsync(); await _log.LogAsync(uid,"Guest document
verification",$"Document {id}: {verified}");
        return RedirectToAction(nameof(Dashboard));
    }
}
```

Application Process

In order to become host, guests must submit at least one property photo together with the property's details(title, description, city, address, property type, price, cleaning cost and service fee). The condition that an application is incomplete without an image is satisfied by server-side validation, which expressly rejects a submission without one.



```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Staybnb.Data;
using Staybnb.Models;
using Staybnb.Services;
using Staybnb.ViewModels;

namespace Staybnb.Controllers;

[Authorize(Roles="Guest")]
public class HostApplicationController : Controller
{
   private readonly ApplicationDbContext _db; private readonly
UserManager<ApplicationUser> _users; private readonly IActivityLogger _log; private readonly
IWebHostEnvironment _env;
   public HostApplicationController(ApplicationDbContext db, UserManager<ApplicationUser>
users,IActivityLogger log,IWebHostEnvironment
env){_db=db;_users=users;_log=log;_env=env;}
   [HttpGet] public IActionResult Apply()=>View(new HostApplicationVm());
   [HttpPost, ValidateAntiForgeryToken]
   public async Task<IActionResult> Apply(HostApplicationVm vm)
   {
      if(vm.Image==null || vm.Image.Length==0)
ModelState.AddModelError(nameof(vm.Image),"At least one property image is required.");
      if(!ModelState.IsValid) return View(vm);
      var userId=_users.GetUserId(User)!; var dir=Path.Combine(_env.WebRootPath,"uploads");
      Directory.CreateDirectory(dir); var
name=Guid.NewGuid()+Path.GetExtension(vm.Image!.FileName); var
path=Path.Combine(dir,name);
      using(var stream=System.IO.File.Create(path)) await vm.Image.CopyToAsync(stream);
```

```
        _db.HostApplications.Add(new
HostApplication{ApplicantId=userId,Title=vm.Title,Description=vm.Description,City=vm.City,
Address=vm.Address,PropertyType=vm.PropertyType,PricePerNight=vm.PricePerNight,Cleani
ngFee=vm.CleaningFee,ServiceFee=vm.ServiceFee,ImagePath="/uploads/"+name});
        await _db.SaveChangesAsync(); await _log.LogAsync(userId,"Host application
submitted",vm.Title);
        return RedirectToAction("Index","Home");
    }
}



[HttpPost, ValidateAntiForgeryToken]
public async Task<IActionResult> Apply(HostApplicationVm vm)
{
    if (vm.Image == null || vm.Image.Length == 0)
        ModelState.AddModelError(nameof(vm.Image), "At least one property image is
required.");
    if (!ModelState.IsValid) return View(vm);

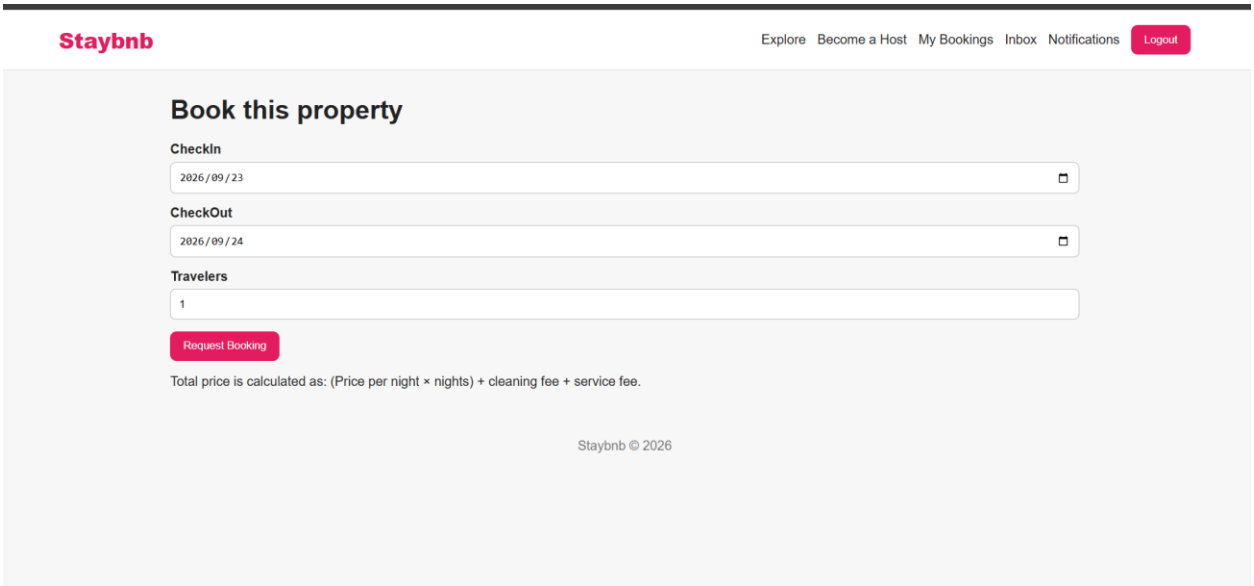
    var userId = _users.GetUserId(User)!;
    var dir = Path.Combine(_env.WebRootPath, "uploads");
    Directory.CreateDirectory(dir);
    var name = Guid.NewGuid() + Path.GetExtension(vm.Image!.FileName);
    var path = Path.Combine(dir, name);
    using (var stream = System.IO.File.Create(path)) await vm.Image.CopyToAsync(stream);

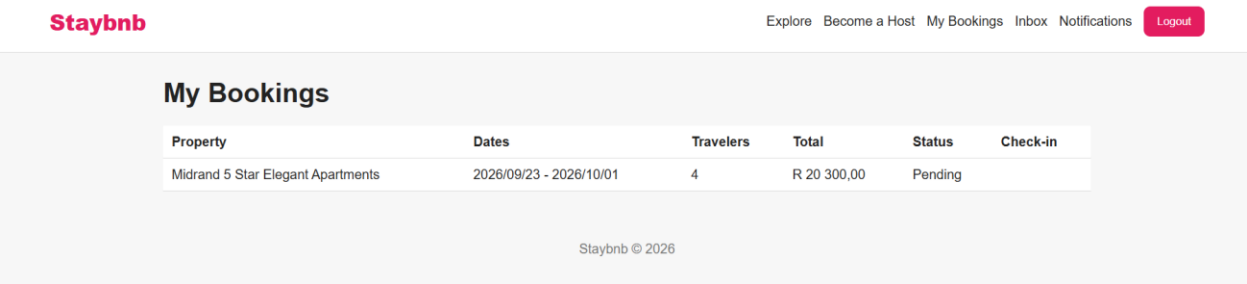
    _db.HostApplications.Add(new HostApplication {
        ApplicantId = userId, Title = vm.Title, Description = vm.Description, City = vm.City,
        Address = vm.Address, PropertyType = vm.PropertyType, PricePerNight =
vm.PricePerNight,
```

```
        CleaningFee = vm.CleaningFee, ServiceFee = vm.ServiceFee, ImagePath = "/uploads/" +
name
    });
    await _db.SaveChangesAsync();
    await _log.LogAsync(userId, "Host application submitted", vm.Title);
    return RedirectToAction("Index", "Home");
}
```

Booking And Check-In

When booking a property, guests choose the dates of check-in and check-out as well as the number of travelers (PricePerNights x Nights) + CleaningFee + ServiceFee is how the toatla cost is determined. A reservation moves through a predetermined status lifecycle: Pending at first, approved or rejected by the host, and then checked in after the host confirms an identification document that the guest uploaded. An intentional attempt to double-book overlapping dates and a check-out date earlier than check-in were both successfully refused during the end-to-end testing.






```
public enum BookingStatus { Pending, Approved, Rejected, CheckedIn }

// On booking creation:
var nights = (checkOut.Date - checkIn.Date).Days;
var total = property.PricePerNight * nights + property.CleaningFee + property.ServiceFee;
var booking = new Booking {
    PropertyId = property.Id, GuestId = guestId, CheckIn = checkIn, CheckOut = checkOut,
Travelers = travelers, TotalPrice = total, Status = BookingStatus.Pending
};
```

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Staybnb.Data;
using Staybnb.Models;
using Staybnb.ViewModels;

namespace Staybnb.Controllers;
[Authorize(Roles="Guest")]
public class BookingController : Controller
{
   readonly ApplicationDbContext _db; readonly UserManager<ApplicationUser> _users;
   public BookingController(ApplicationDbContext db,UserManager<ApplicationUser>
users){_db=db;_users=users;}
   [HttpGet] public async Task<IActionResult> Create(int id){var p=await
_db.Properties.FindAsync(id); if(p==null||!p.IsActive)return NotFound(); return View(new
BookingVm{PropertyId=id});}
   [HttpPost,ValidateAntiForgeryToken]
   public async Task<IActionResult> Create(BookingVm vm)
   {
      var p=await _db.Properties.FindAsync(vm.PropertyId); if(p==null||!p.IsActive)return
NotFound();
      if(vm.CheckOut<=vm.CheckIn) ModelState.AddModelError("", "Check-out must be after
check-in.");
      var overlap=await _db.Bookings.AnyAsync(b=>b.PropertyId==vm.PropertyId &&
b.Status!=BookingStatus.Rejected && vm.CheckIn<b.CheckOut &&
vm.CheckOut>b.CheckIn);
      if(overlap) ModelState.AddModelError("", "The property is not available for those dates.");
      if(!ModelState.IsValid) return View(vm);
      var nights=(vm.CheckOut.Date-vm.CheckIn.Date).Days;
      var total=(p.PricePerNight*nights)+p.CleaningFee+p.ServiceFee;
      var b=new
Booking{PropertyId=p.Id,GuestId=_users.GetUserId(User)!,CheckIn=vm.CheckIn,CheckOut=v
m.CheckOut,Travelers=vm.Travelers,TotalPrice=total};
      _db.Bookings.Add(b); _db.Notifications.Add(new
Notification{UserId=p.HostId,Title="New Booking Request",Message=$"A new booking
request was made for {p.Title}."});
      await _db.SaveChangesAsync(); return RedirectToAction(nameof(MyBookings));
   }
   public async Task<IActionResult> MyBookings()
   {
      var uid = _users.GetUserId(User)!;
      ViewBag.ReviewedPropertyIds = (await _db.Reviews.Where(r => r.ReviewerId ==
uid).Select(r => r.PropertyId).ToListAsync()).ToHashSet();
      return View(await _db.Bookings.Include(b => b.Property).Where(b => b.GuestId ==
uid).OrderByDescending(b => b.Id).ToListAsync());
   }
```

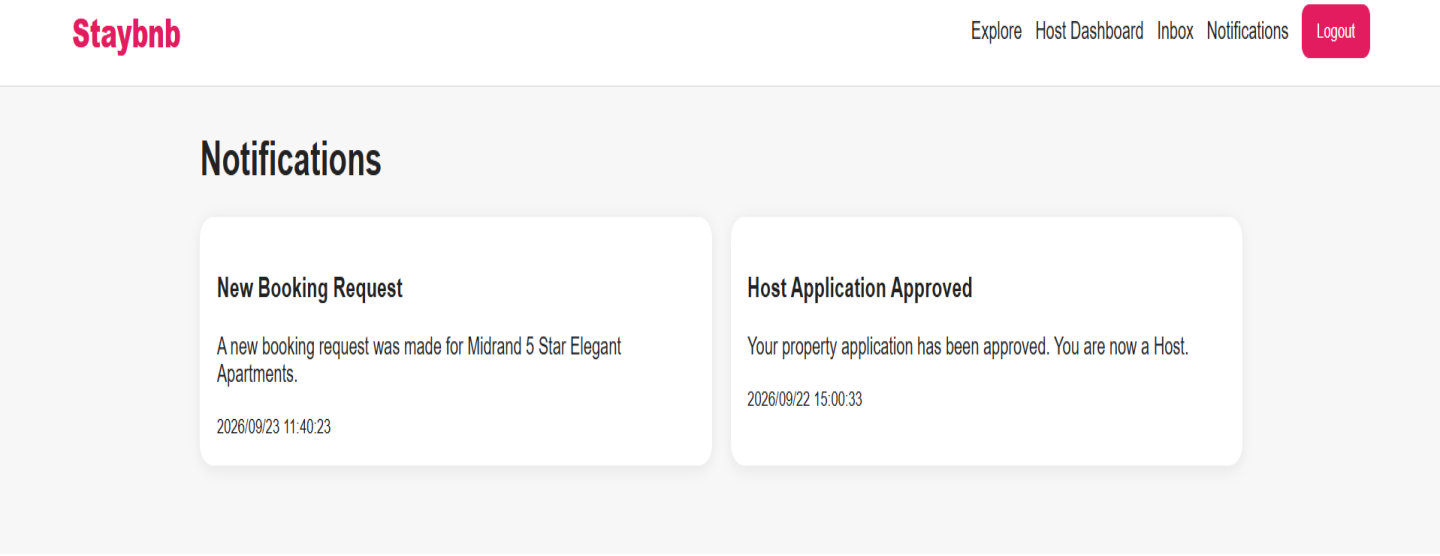
```
   [HttpGet] public async Task<IActionResult> CheckIn(int id){var b=await
_db.Bookings.Include(x=>x.Property).FirstOrDefaultAsync(x=>x.Id==id&&x.GuestId==_users.
GetUserId(User)&&x.Status==BookingStatus.Approved); if(b==null)return NotFound(); return
View(b);}
   [HttpPost,ValidateAntiForgeryToken] public async Task<IActionResult> CheckIn(int
id,IFormFile? document)
   {
      var b=await
_db.Bookings.FirstOrDefaultAsync(x=>x.Id==id&&x.GuestId==_users.GetUserId(User)&&x.St
atus==BookingStatus.Approved); if(b==null)return NotFound();
      if(document==null||document.Length==0){ModelState.AddModelError("","ID or Passport
document is required."); return View(b);}
      var
dir=Path.Combine(Directory.GetCurrentDirectory(),"wwwroot","uploads");Directory.CreateDire
ctory(dir);var name=Guid.NewGuid()+Path.GetExtension(document.FileName);using var
s=System.IO.File.Create(Path.Combine(dir,name));await document.CopyToAsync(s);
      _db.GuestDocuments.Add(new
GuestDocument{BookingId=id,DocumentPath="/uploads/"+name});await
_db.SaveChangesAsync(); return RedirectToAction(nameof(MyBookings));
   }
   [HttpGet]
   public async Task<IActionResult> AddReview(int bookingId)
   {
      var uid = _users.GetUserId(User)!;
      var b = await _db.Bookings.Include(x => x.Property)
         .FirstOrDefaultAsync(x => x.Id == bookingId && x.GuestId == uid && x.Status ==
BookingStatus.CheckedIn);
      if (b == null) return NotFound();
      var already = await _db.Reviews.AnyAsync(r => r.PropertyId == b.PropertyId &&
r.ReviewerId == uid);
      if (already) return RedirectToAction(nameof(MyBookings));
      ViewBag.PropertyTitle = b.Property!.Title;
      return View(new Review { PropertyId = b.PropertyId });
   }

   [HttpPost, ValidateAntiForgeryToken]
   public async Task<IActionResult> AddReview(Review review)
   {
      var uid = _users.GetUserId(User)!;
      if (review.Rating < 1 || review.Rating > 5) ModelState.AddModelError("", "Rating must be
between 1 and 5.");
      ModelState.Remove("ReviewerId");
      var ownsStay = await _db.Bookings.AnyAsync(x => x.PropertyId == review.PropertyId
&& x.GuestId == uid && x.Status == BookingStatus.CheckedIn);
      if (!ownsStay) return Forbid();
      if (!ModelState.IsValid) return View(review);
```

```
        var already = await _db.Reviews.AnyAsync(r => r.PropertyId == review.PropertyId &&
r.ReviewerId == uid);
        if (already) return RedirectToAction(nameof(MyBookings));
        review.ReviewerId = uid;
        review.Id = 0;
        _db.Reviews.Add(review);
        await _db.SaveChangesAsync();
        return RedirectToAction(nameof(MyBookings));
    }
}
```

Notifications and Messaging

A database-backend inbox allows registered users to send and receive messages. All of a user's received messages are marked as read when they open their inbox. When a user opens the notification page, all entries are marked as read and both read states are stored in the databaae. This is how a parallel notification system notifies users of events like a new message or a change in the status of a booking

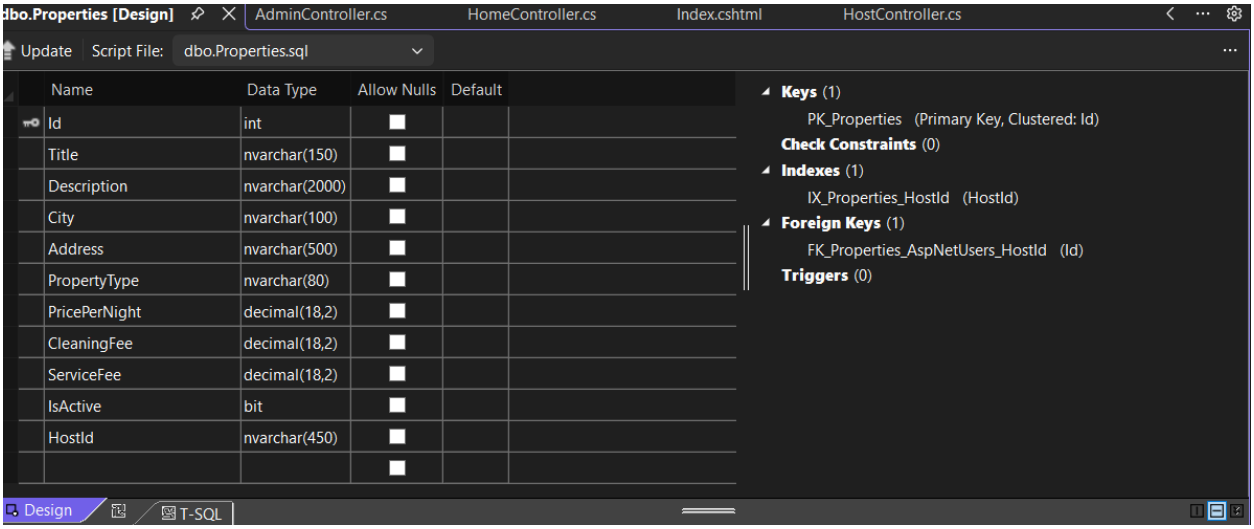


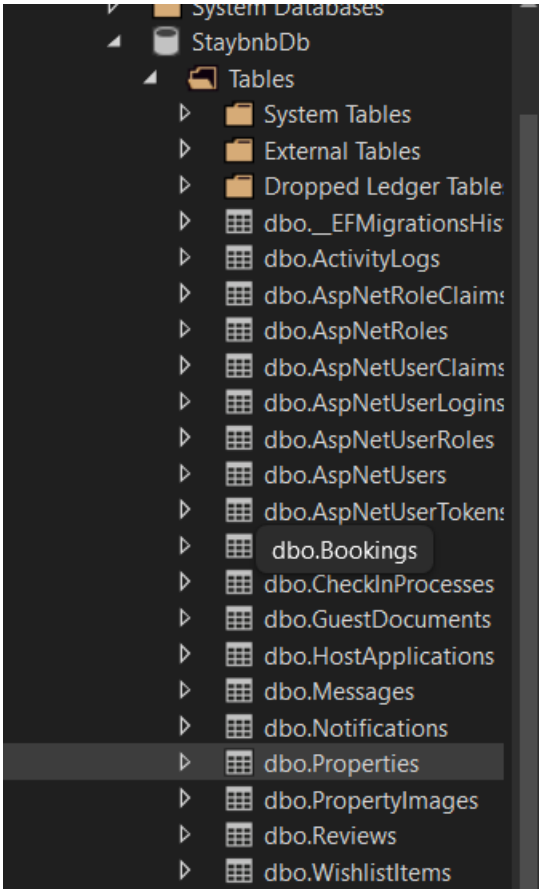

```
public async Task<IActionResult> Inbox()
{
    var id = _users.GetUserId(User)!;
    var messages = await _db.Messages.Include(x => x.Sender)
        .Where(x => x.RecipientId == id).OrderByDescending(x => x.SentAt).ToListAsync();
    foreach (var m in messages) m.IsRead = true;
    await _db.SaveChangesAsync();
    return View(messages);
}

public async Task<IActionResult> Notifications()
{
    var id = _users.GetUserId(User)!;
    var n = await _db.Notifications.Where(x => x.UserId == id)
        .OrderByDescending(x => x.CreatedAt).ToListAsync();
    foreach (var x in n) x.IsRead = true;
    await _db.SaveChangesAsync();
    return View(n);
}
```

Data and Architechture

Each entity(Proptery, Booking, HostApplication, CheckInProcess, GusetDocument, Message, Notification, ActivityLog, Review, and WishlistItem) is mapped through ApplicationDbContext in order to contrast the data model using Entity Framework Core Code-First migrations against SQL Server. A clear audit trail is provided by the integration of activity logging into important application operations, such as host application decisions, booking status changes, property toggles, and document verification. Two flaws were found and fixed during development: a Razor parsing problem across several views due to a missing line break following the @model directive, and a missing NuGet package reference for ASP.NET Core Identity's default user interface.



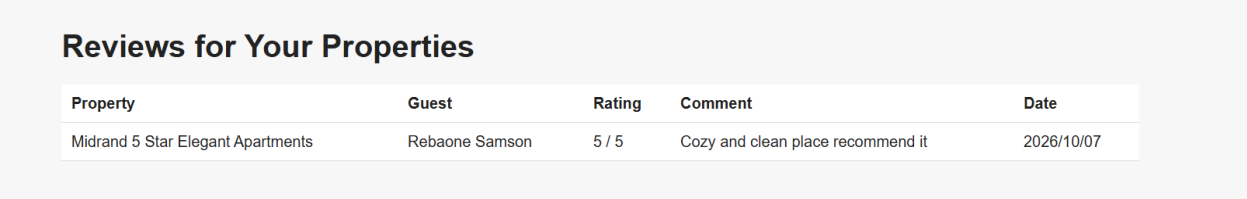



```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }

    public DbSet<Property> Properties => Set<Property>();
    public DbSet<PropertyImage> PropertyImages => Set<PropertyImage>();
    public DbSet<HostApplication> HostApplications => Set<HostApplication>();
    public DbSet<Booking> Bookings => Set<Booking>();
    public DbSet<CheckInProcess> CheckInProcesses => Set<CheckInProcess>();
    public DbSet<GuestDocument> GuestDocuments => Set<GuestDocument>();
    public DbSet<Message> Messages => Set<Message>();
    public DbSet<Notification> Notifications => Set<Notification>();
    public DbSet<ActivityLog> ActivityLogs => Set<ActivityLog>();
    public DbSet<Review> Reviews => Set<Review>();
    public DbSet<WishlistItem> WishlistItems => Set<WishlistItem>();
```

```
migrationBuilder.CreateTable(
    name: "ActivityLogs",
    columns: table => new {
        Id = table.Column<int>(type: "int", nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        UserId = table.Column<string>(type: "nvarchar(max)", nullable: true),
        Action = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: false),
        Details = table.Column<string>(type: "nvarchar(1000)", maxLength: 1000, nullable: false),
        CreatedAt = table.Column<DateTime>(type: "datetime2", nullable: false)
    });
```

REVIEWS



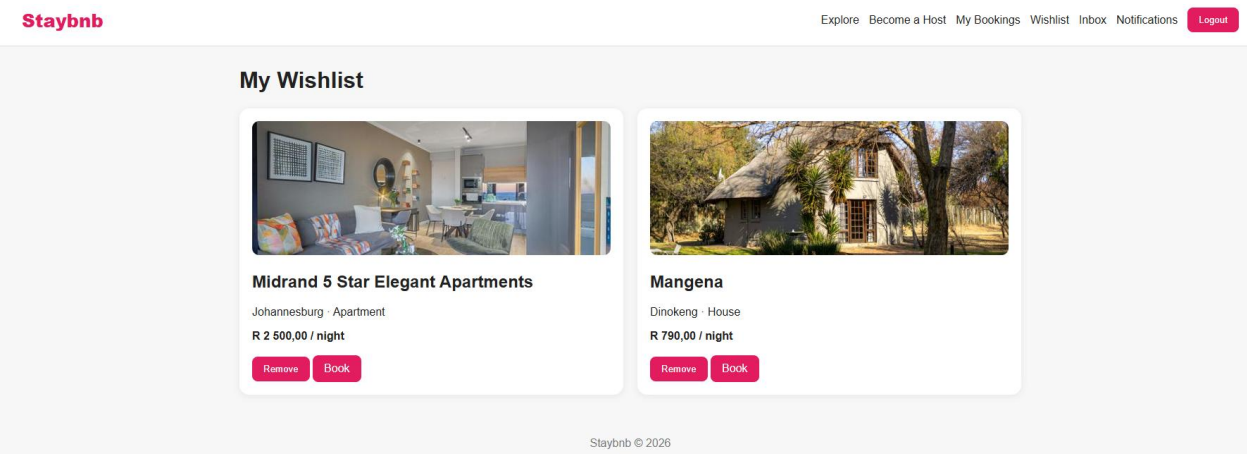

```
@model IEnumerable<Review>
 <h1>Reviews for Your Properties</h1> <table class="table">
<tr><th>Property</th><th>Guest</th><th>Rating</th><th>Comment</th><th>Date</th></tr>
@foreach(var r in Model) { <tr> <td>@r.Property?.Title</td>
<td>@r.Reviewer?.FullName</td> <td>@r.Rating / 5</td> <td>@r.Comment</td>
<td>@r.CreatedAt.ToShortDateString()</td> </tr> } </table>
```

ADDITIONAL FEATURES

The following features I added to the original specification to increase the capability of the platform: a wishlist feature that allows any authenticated user to save properties for later with a single toggle; a Host Earnings page that aggregates confirmed booking revenue; a Review system that lets guests rate and comment on a property after a completed (CheckedIn) stay, with and average rating shown on both the Host Dashboard and each property's listing card; and Admin Dashboard statistics that cover total users, active properties, platform revenue, approved

Host.

WISHLISTITEM




```
public class WishlistItem
{
    public int Id { get; set; }
    public string UserId { get; set; } = "";
    public ApplicationUser? User { get; set; }
    public int PropertyId { get; set; }
    public Property? Property { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```




```
public class Review
{
    public int Id { get; set; }
    public int PropertyId { get; set; }
    public Property? Property { get; set; }
    [Required] public string ReviewerId { get; set; } = "";
```

```
public ApplicationUser? Reviewer { get; set; }
[Range(1, 5)] public int Rating { get; set; }
[Required, MaxLength(1000)] public string Comment { get; set; } = "";
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

REFERENCE

Microsoft (2024) Introduction to Identity on ASP.NET Core. Available at: https://learn.microsoft.com/aspnet/core/security/authentication/identity (Accessed: 13 September 2026).

Microsoft (2024) Entity Framework Core documentation. Available at: https://learn.microsoft.com/ef/core/ (Accessed: 9 August 2026).

Microsoft (2024) Overview of ASP.NET Core MVC. Available at: https://learn.microsoft.com/aspnet/core/mvc/overview (Accessed: 13 October 2026).